Virtual Networking Basics (SDN Principles)

Virtual Networking Basics (SDN Principles)

- Cloud Networking Foundations Bootcamp
- Introduction of My Self

Why This Topic Matters

Why Learn SDN First?

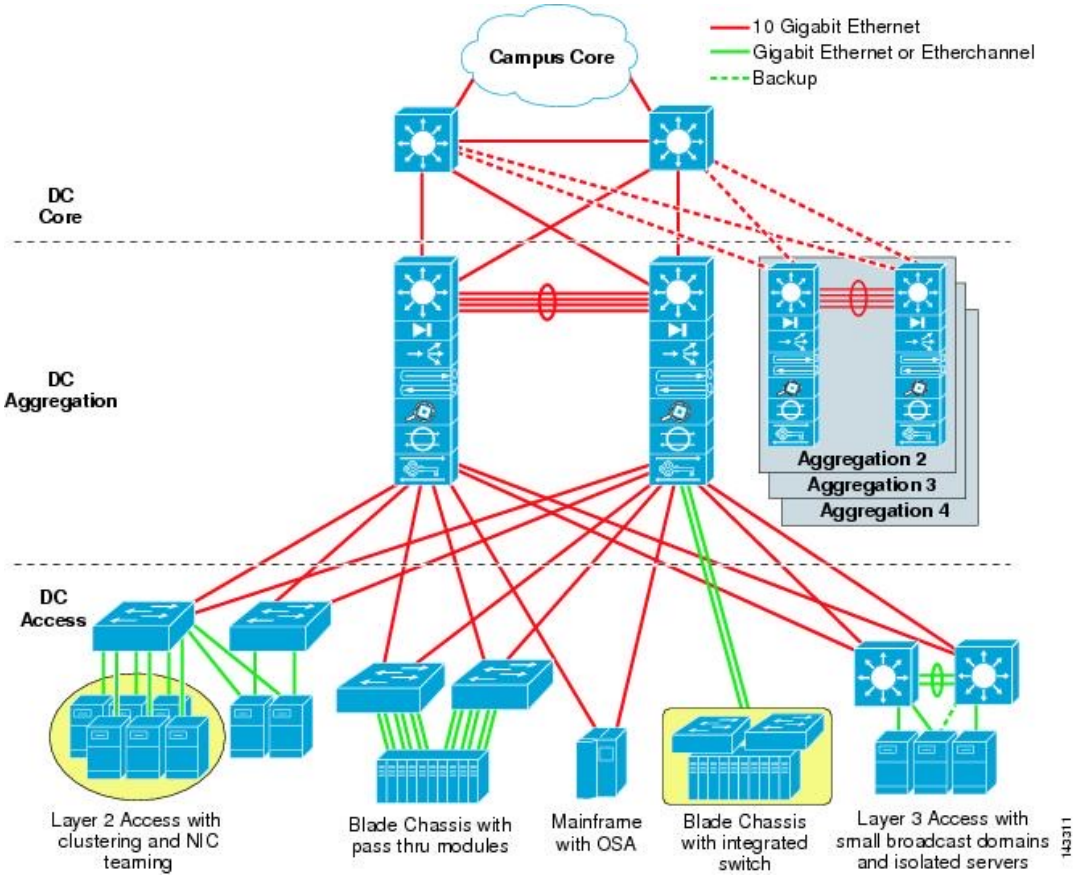
- Cloud networking ≠ traditional networking
- Everything is software-defined
- You manage policies, not hardware

Required before learning:

- Amazon VPC
- Azure Virtual Network
- Google Cloud VPC

Traditional Networking Model

- **Traditional Data Center Networking**
  - Physical routers & switches
  - VLAN configuration
  - Device-by-device setup
  - Manual CLI changes
  - Hardware-dependent scaling



Limitations of Traditional Networking

Challenges

- Slow provisioning
- Risky configuration changes
- Limited scalability
- Hardware lifecycle dependency
- Vendor lock-in

What is Software-Defined Networking?

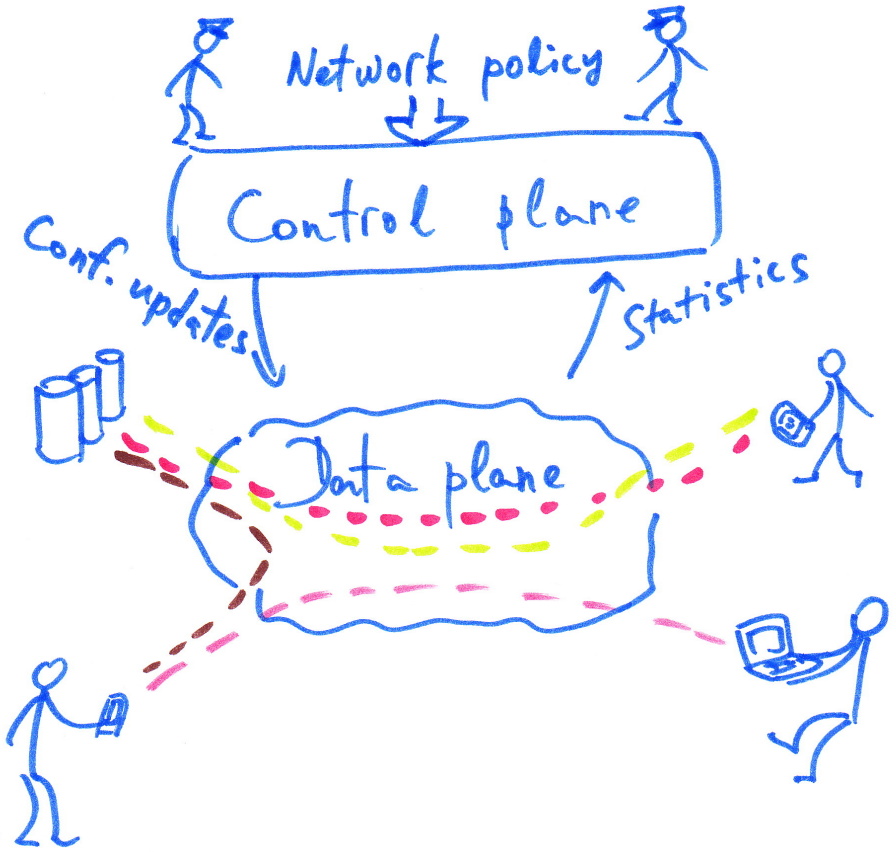
SDN separates the control plane from the data plane and centralizes control in software.

- Logic is centralized
- Forwarding is distributed
- Network is programmable via API

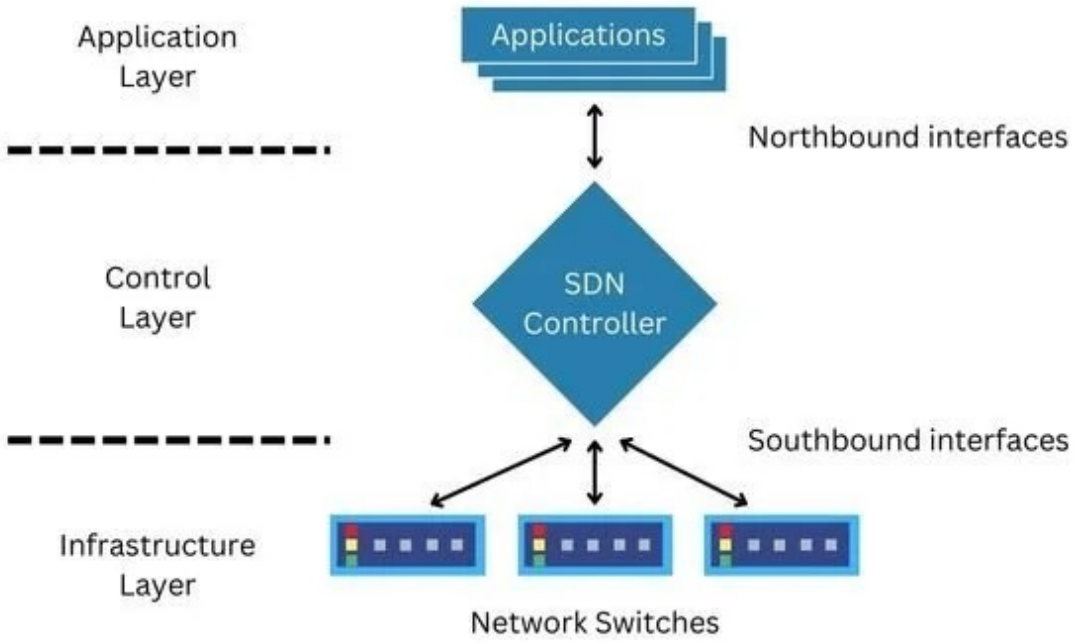
Control Plane vs Data Plane

Two Fundamental Components

- **Control Plane**
  - Makes routing decisions
  - Programs forwarding rules
  - Maintains topology
- **Data Plane**
  - Forwards packets
  - Applies rules
  - Moves traffic



Architecture Diagram



How Cloud Providers Use SDN

Cloud Networking is SDN : Its makes cloud networks **smart, flexible, automated, and safe**. It's like having a digital traffic controller for millions of users at once.

Examples:Amazon VPC, Azure Virtual Network,Google Cloud VPC

What you define:

- Subnets
- Route tables
- Security policies

What happens internally:

- SDN controller configure Switches/Server
- Virtual switching enforced
- Traffic encapsulated across physical network

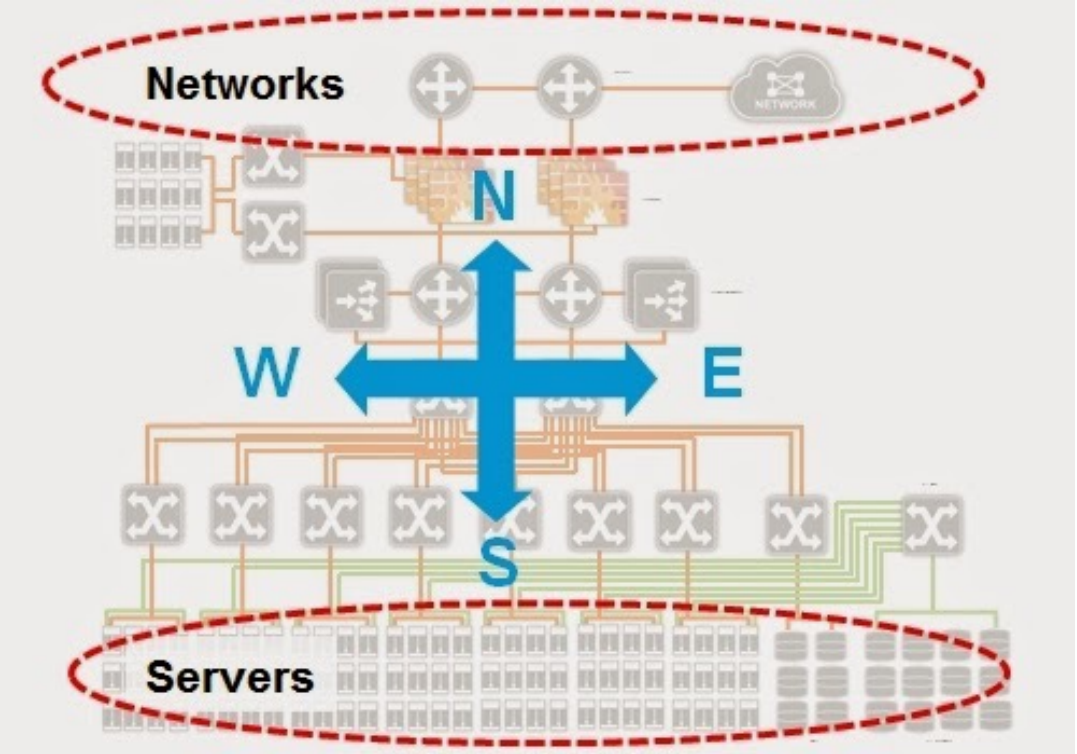
East-West vs North-South Traffic

North-South Traffic

- Incoming / outgoing traffic
- Internet ↔ Cloud

East-West Traffic

- Internal service-to-service communication
- Web ↔ App ↔ DB
- Microservices traffic



Overlay and Underlay (Cloud Building Blocks)

Overlay Network

Overlay technology is a networking approach where you build a **virtual network on top of an existing physical network (underlay)**. The overlay creates logical connections between nodes, while the underlay just handles basic packet transport.

Example:

Original Packet → Encapsulated → Sent via Underlay → Decapsulated

Underlay Network

Underlay technology is the **physical or base network infrastructure** that actually carries packets from one point to another. It's the "real" network—routers, switches, links, IP routing—on top of which overlays are built.

Example:

Source IP → Routing decision → Forward packet → Destination IP

No encapsulation logic, no virtualization—just pure packet forwarding.

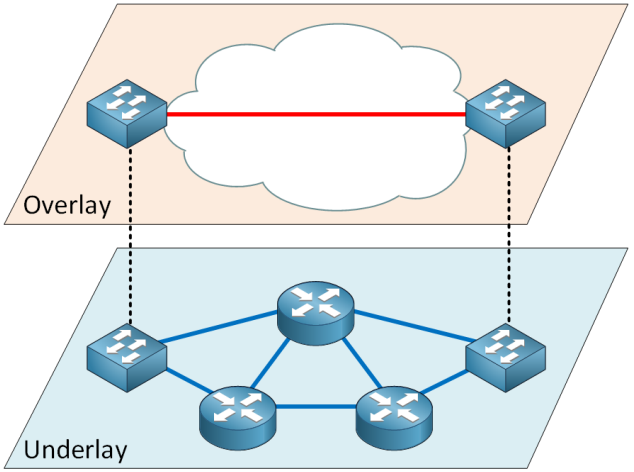
Overlay vs Underlay Networking

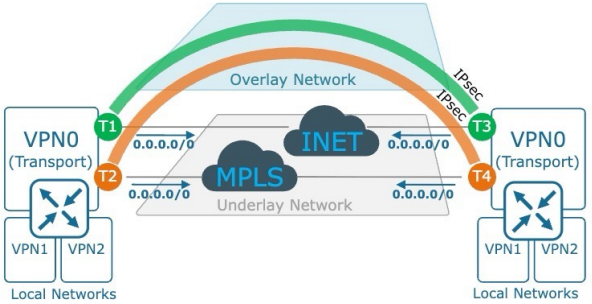
Underlay Network

- Physical infrastructure
- Fiber, routers, spine-leaf topology
- Managed by cloud provider

Overlay Network

- Virtual networks (VPC/VNet)
- Logical subnets
- Virtual routers





NAT (SNAT / DNAT Concepts)

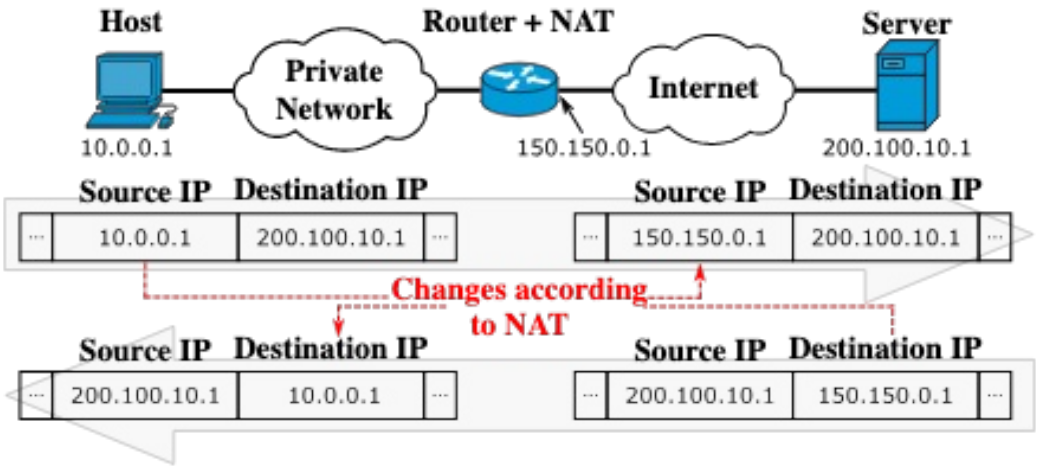
What Is NAT?

Network Address Translation

- Modifies IP addresses in packet header
- Enables private IP usage
- Conserves public IPv4 space
- Two Types:
  - SNAT
  - DNAT

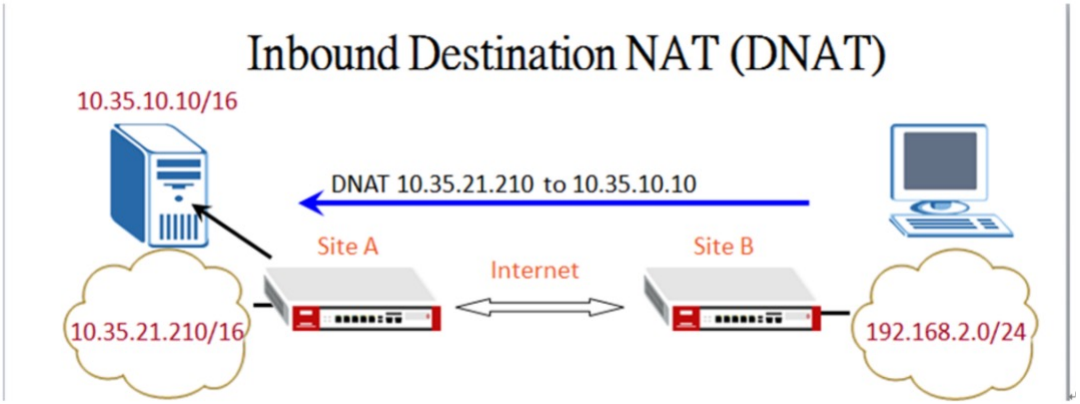
Source NAT (SNAT)

- Changes source IP
- Used for outbound internet access
- Private → Public translation
- Example:
  Private VM → NAT Gateway → Internet



Destination NAT (DNAT)

- Changes destination IP
- Used for inbound access
- Often via load balancer
- Example:
  Public IP → Internal server



NAT Design Considerations

- Port exhaustion risk
- High availability NAT design
- NAT is not security
- Logging is important
- Key Takeaway: NAT hides IPs, but does not replace firewalling.

Public vs Private IP

Public IP

- Globally routable
- Reachable over internet
- Used for:
  - Load balancers
  - Bastions
  - Public APIs
- Risk:
  - Increased attack surface

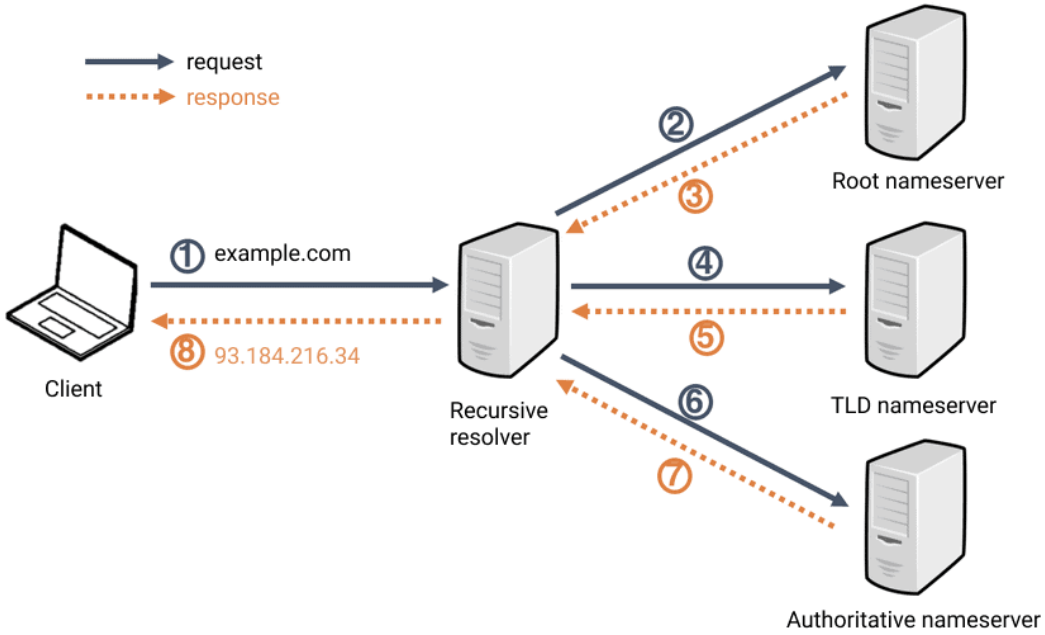
Private IP

- RFC1918 ranges
- Not internet routable
- Default in most cloud deployments
- Used for internal communication

DNS Architecture in Cloud

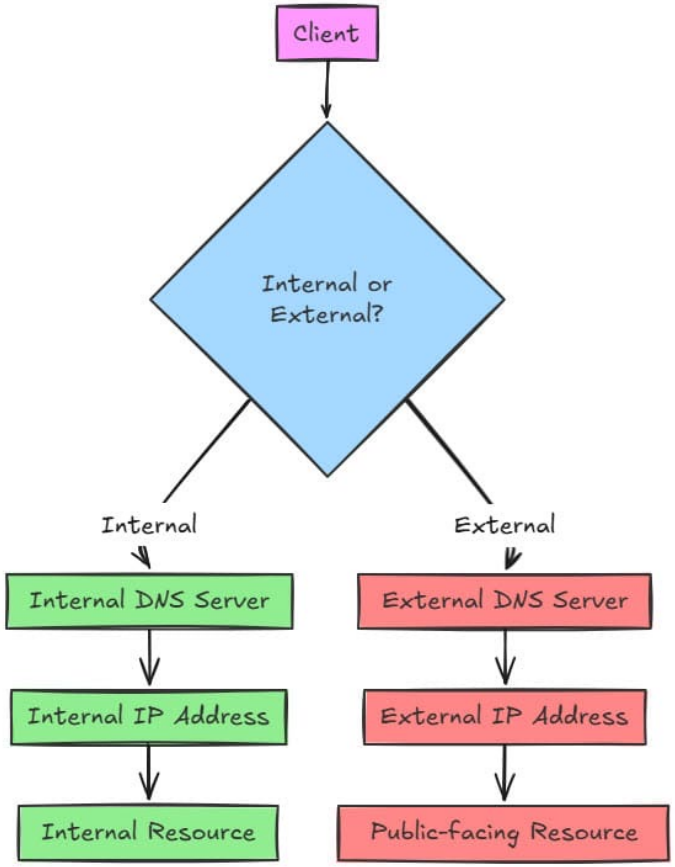
DNS Basics

- DNS Resolves: Name → IP address
- Components:
  - Recursive resolver
  - Authoritative server



Public vs Private DNS

- Public DNS:
  - Internet-facing domains
- Private DNS:
  - Internal service discovery
  - Supports: Split-horizon DNS



Hybrid DNS Design

A setup where your cloud and on-premises systems can resolve each other's domain names using DNS.

"Hybrid DNS = Cloud + On-prem talking to each other using DNS."

Think of two offices:

- Office A = Cloud
- Office B = On-prem

Each has its own phone directory.

- 👉 Hybrid DNS is like **connecting both directories**, so:
  - People in Office A can call people in Office B
  - And vice versa

Load Balancing (L4 vs L7)

Load Balancing?

Load balancing = distributing incoming traffic across multiple servers so no single server gets overloaded.

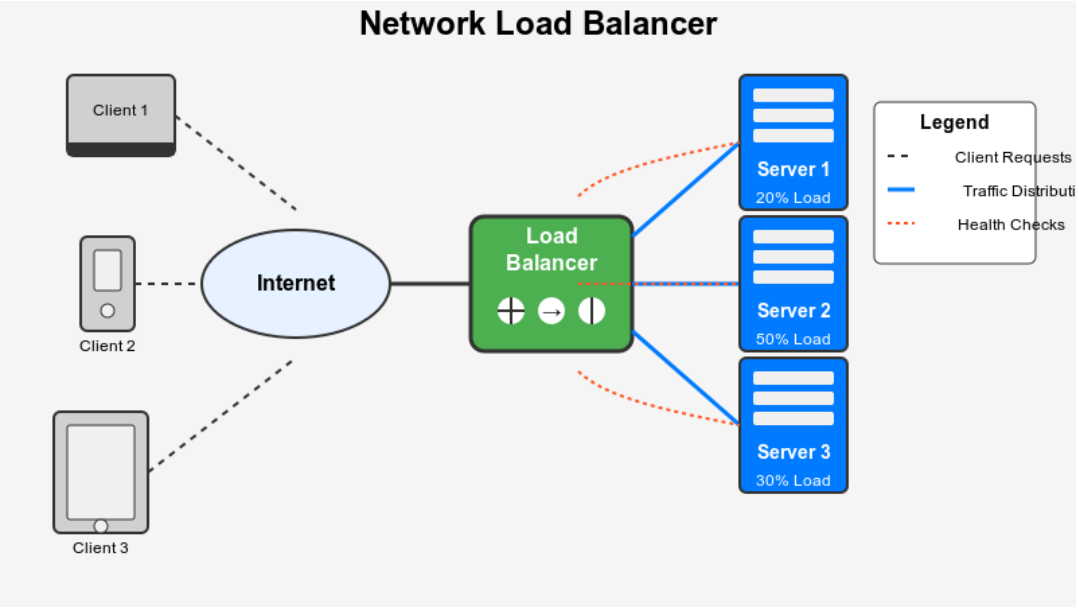
Example :

“A restaurant with 1 waiter vs 5 waiters”

- 1 waiter → slow service ❌
- 5 waiters + manager assigning tables → fast service ✅

- 👉 Load balancer = **manager assigning customers**

User → Load Balancer → Server 1 / Server 2 / Server 3



Layer 4 Load Balancer

Layer 4 load balancing distributes traffic based on IP address and port (TCP/UDP), without looking inside the actual data.

Load balancing decisions are made using:

- Source IP
- Destination IP
- Source port
- Destination port
- Protocol (TCP/UDP)

Example

User sends request:

TCP 10.1.1.5:5000 → 20.1.1.10:80

Load balancer decides:

- Send to Server A or B
- based only on connection info

Layer 7 Load Balancer

Layer 7 load balancing distributes traffic based on application data (like URL, headers, cookies), not just IP and port.

Example:
User → Load Balancer → Decision based on URL → Specific server

/api → Server A
/images → Server B
/login → Server C

High Availability

Active-Active

Multiple systems are active at the same time and share the load.

User → Load Balancer → Server A (active)

→ Server B (active)

Example:

“Two chefs cooking together in the kitchen”

- Orders are split between both
- Faster service
- If one leaves → other continues

Active-Passive

One system is active, the other is standby (idle) until failure happens.

User → Active Server

↓ (fails)

Passive Server takes over

Example:

“One chef cooking, another waiting nearby”

- Backup only steps in if main chef fails

N+1 Redundancy

M+1 redundancy = you need M components to run the system, and you add 1 extra as backup.

Load Balancer → Server 1
→ Server 2
→ Server 3
→ Server 4
→ Server 5 (backup)

Zonal vs Regional vs DR in Cloud

Zonal Resources

- **Zonal = everything in one Availability Zone**

  In cloud (like AWS, Azure or GCP):

  - A **Region** = geographic area (e.g., Mumbai)
  - Inside it → multiple **Availability Zones (AZs)**

Example:

- AWS Mumbai Region
  - AZ1
  - AZ2
  - AZ3

Regional Services

Resources are distributed across multiple Availability Zones within a region

In cloud platforms like AWS:

- A **region** = one geographic area
- Inside it → multiple isolated data centers (AZs)

Example: AWS Mumbai Region has multiple AZs.

DR

Cross-region DR = running your system in multiple regions so it survives even a full region outage.

Regional architecture protects you from:

- AZ failure ✅

But not from:

- Entire region outage ❌
  (rare, but it happens)

👉 Cross-region DR solves that.

API-Driven Networking

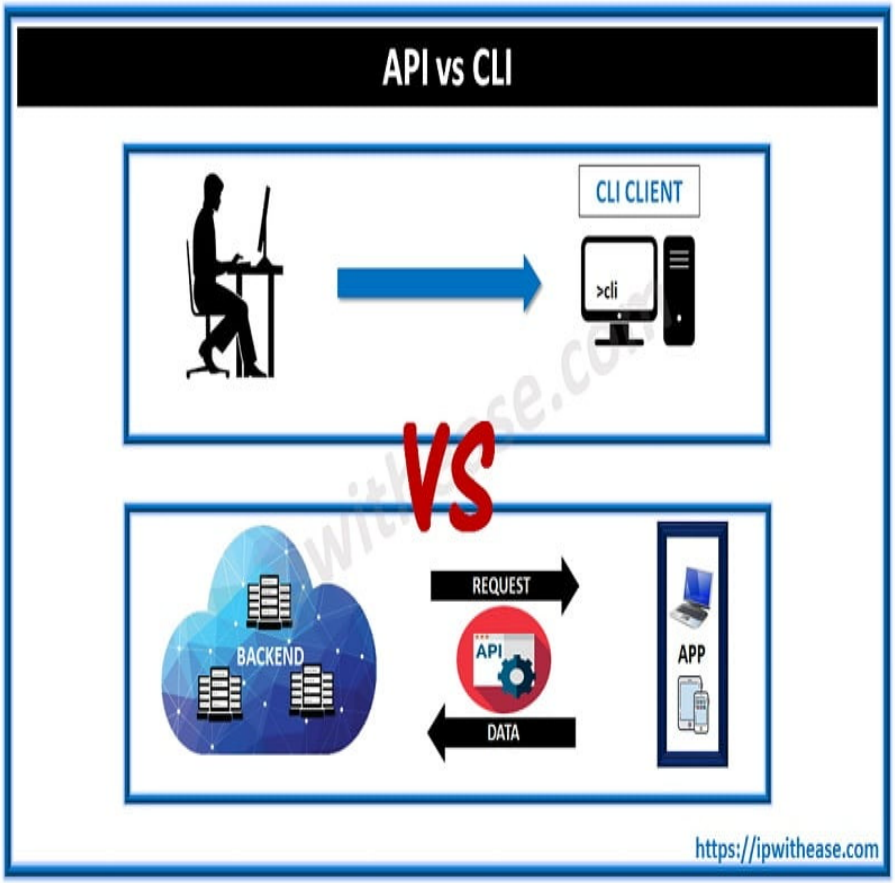
Cloud uses Networking as Code

Instead of CLI on routers:

- API calls - Talk to network devices or cloud networks using software commands.
- Terraform/ARM templates/CloudFormation - Special files that **describe your network setup** so it can be created automatically.

Benefits:

- Repeatable
- Version controlled
- Automated
- Auditable



END

Multi-Tier Application Network Design (4 Slides)

Classic 3-Tier Model

- Web Tier (Public)
- App Tier (Private)
- Database Tier (Isolated)

Traffic Flow

- User → LB → Web → App → DB
- North-South:
  User traffic
- East-West:
  Service communication

Modern Evolution

- Microservices
- Service mesh
- API gateways
- Zero-trust networking

Design Principles

- Segmentation by tier
- Least privilege access
- Separate environments
- Plan for scale
- Key Takeaway:
- Network architecture should reflect application architecture.

Global Expansion Architecture Challenge

- 🌍 A company requires:
- HQ On-Prem Data Center
- 2 Cloud Regions
- Dev, Test, Prod environments
- High Availability (Multi-AZ)
- Internet-facing web application
- Secure database layer
- Hybrid connectivity (VPN or Direct Connect)
- Disaster Recovery capability

- **Next: CIDR & IP Planning at Scale**
- Now that networking is software:
- Address design becomes critical
- Overlapping IP breaks architecture
- Scaling requires planning

CIDR Fundamentals

- **Understanding CIDR Notation**
- IPv4 = 32-bit address
- CIDR format: IP / Prefix
  - Example: 10.0.0.0/16
- Prefix = number of network bits
- Remaining bits = host addresses
- Smaller prefix number = larger network
- **Common Examples**
- /24 → 256 IPs (254 usable)
- /16 → 65,536 IPs
- /20 → 4,096 IPs

Private IP Ranges & Overlapping Problem

- **RFC1918 Private Address Space**
- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16
- These are:
- Not publicly routable
- Used inside cloud networks
- ⚠️ **Overlapping CIDR = Architecture Failure**
- Problems caused:
- VPC peering fails
- VPN routing conflicts
- Hybrid connectivity breaks
- Route ambiguity

IP Planning for Multi-Region Design

- **Planning for Scale**
- Bad Approach:
- Random CIDR per VPC
- Good Approach:
- Hierarchical IP design

Example:

Region	CIDR
US-East	10.0.0.0/16
US-West	10.1.0.0/16
Europe	10.2.0.0/16

Design Exercise (Interactive)

- **Scenario**
- Company Requirements:
- 3 Regions
- 2 Availability Zones per region
- Dev, Test, Prod environments
- Future on-prem VPN connection
- **Design:**
- Overall CIDR block?
- Region allocation?
- Environment segmentation?
- Room for future growth?

Key Takeaways

•CIDR defines scalability
•Avoid overlapping networks
•Use hierarchical allocation
•Plan for hybrid + multi-region

Route Tables & Route Propagation (4 Slides)

- **Route Tables Fundamentals**
- **What Is a Route Table?**
- Defines how traffic leaves a subnet
- Contains:
  - Destination CIDR
  - Target (gateway, peering, NAT, etc.)
- Uses **Longest Prefix Match**
- Every subnet must associate with a route table
- **Speaker Notes:**
  Explain that routing is decision logic.
  Most specific route wins.
  Example:
- 10.0.0.0/16 → local
- 10.0.1.0/24 → firewall
  Traffic to 10.0.1.10 follows /24 route.

Default Routes & Internet Access

- **Default Route**
- 0.0.0.0/0 → “Everything else”
- Used for:
- Internet access
- Outbound connectivity
- NAT routing
- Targets may include:
- Internet Gateway
- NAT Gateway
- Firewall appliance
- **Speaker Notes:**
  If no default route → no internet access.
  If wrong target → traffic blackholes.
-

Route Propagation (Conceptual)

- **Static vs Dynamic Routes**
- Static:
- Manually defined
- Dynamic:
- Learned via BGP
- Used in hybrid connectivity
- Propagation Used In:
- VPN connections
- Transit gateways
- ExpressRoute / Interconnect

Common Routing Problems

- **Architecture Issues**
- Overlapping CIDR
- Asymmetric routing
- Missing return route
- Blackhole routes
- Key Principle:
- Routing must work both directions.

Default Routes & Blackholing (3 Slides)

Understanding Default Routes

- **0.0.0.0/0**
- Matches all unspecified traffic
- Usually points to:
  - Internet gateway
  - NAT
  - Firewall
- Used in:
- Public subnets
- Private egress subnets

What Is Blackholing?

- **Blackhole Route**
- Route target no longer exists
- Traffic silently dropped
- No explicit error
- Common Causes:
- Deleted NAT
- Removed peering
- Misconfigured gateway

Prevention Best Practices

- Monitor route table changes
- Validate targets exist
- Use infrastructure as code
- Test failover paths
- Key Idea:
- Routing errors break entire architectures.

Distributed Security Enforcement

Security in SDN

- Policy enforced at hypervisor
- Distributed firewall model
- Stateful filtering possible
- Micro-segmentation enabled

Interactive Exercise

Scenario Discussion

- Company needs:
  - Dev, Test, Prod isolation
  - Web + DB in each
  - No physical hardware

Questions:

- Where is control plane?
- Where is data plane?
- How is segmentation enforced?
- What programs forwarding rules?

Common Misconceptions

Misconceptions

- ❌ “Cloud networking is simpler.”
  ✓ It’s abstracted.
- ❌ “There are no routers.”
  ✓ There are virtual routers.
- ❌ “Security groups are just firewalls.”
  ✓ They are distributed stateful policies.

Key Takeaways

Summary

- SDN separates control and data planes
- Cloud networking is software-defined
- Overlay runs on provider underlay
- Policies are API-driven
- Everything is programmable